Image Denoising and Enhancement Pipeline

Suvrojit Ghosh

Abstract

This project focuses on the development of an image processing pipeline to restore degraded images by removing noise and enhancing details. Specifically, two distinct approaches were utilized for two separate image types: a landscape image and a graphic (Iron-Man). Through the application of bilateral filtering, Non-Local Means denoising, contour-based masking, and gamma correction, the noisy inputs were significantly improved. In the context of Aerial Robotics Kharagpur (ARK), these preprocessing techniques are highly scalable and critical for robust downstream computer vision tasks, such as reliable object detection and environmental mapping under poor visual conditions.

I. INTRODUCTION

The objective of this project is to process and recover clear visual information from images corrupted by noise and artifacts. The approach involves tailoring the computer vision pipeline to the specific characteristics of the image. For continuous-tone scenery, edge-preserving smoothing and contrast adjustments were prioritized. For structured graphical images, contour-based extraction and localized denoising were utilized. Initial simplistic methods, such as basic blurring, often fail by washing out critical high-frequency details; therefore, more advanced multi-pass algorithms and selective masking were employed.

II. PROBLEM STATEMENT

The primary problem is to effectively denoise two different degraded image sets located in separate directories while retaining their underlying features:

- **Task 1 (Scenery):** A 65 KB noisy landscape image named noisy.jpg requires smoothing of color jitter without losing the crispness of the environmental edges.
- **Task 2 (Iron-Man):** A 175 KB noisy graphic named iron_man_noisy.jpg requires the separation of the main subject from severe background noise before refinement.

The goal is to generate clean outputs (enhanced_landscape.jpg and iron_man_clean.jpg) that are visually clear and structurally intact.

III. RELATED WORK

Standard approaches for image denoising heavily rely on foundational spatial filters. Techniques like Gaussian Blurring and Median Blurring are frequently implemented to tackle high-frequency noise. However, these basic convolutional approaches often fail to differentiate between noise and actual image edges, resulting in a loss of structural integrity.

IV. INITIAL ATTEMPTS

Initial attempts at solving the noise problem involved using standard filtering techniques.

- **Scenery:** The initial solution considered was a Median Blur. However, this was rejected because it tends to blur everything uniformly rather than smoothing specific pixels, which would destroy the crisp edges of the landscape.
- **Iron-Man:** Basic thresholding on the RGB channels was an initial thought, but the noise levels required structural isolation (contours) to prevent the subject from being heavily fragmented.

V. FINAL APPROACH

The final approaches were implemented via two Python scripts, tailored to each image's properties.

1. Scenery Enhancement (noisy_clean.py)

- **Strong Bilateral Filter:** The image is first passed through cv2.bilateralFilter(image, 9, 75, 75). This smooths pixels with similar colors while keeping edges crisp, avoiding the uniform blurring of a median filter.
- **Multi-pass Non-Local Means:** To clean up remaining color jitter without creating a plastic look, a light-strength cv2.fastNlMeansDenoisingColored is applied with parameters 10, 10, 7, 21.
- **Detail Enhancement:** Features are made visible without creating glitchy artifacts using cv2.detailEnhance(denoised_fine, sigma_s=10, sigma_r=0.15).
- **Final Contrast Boost:** A Gamma Correction is applied using a Look-Up Table (LUT) with gamma = 1.2 to pull mountain features out of the shadows.

2. Iron-Man Extraction and Denoising (Iron-man.py)

- **Binarization:** The image iron_man_noisy.jpg is read using cv2.imdecode and converted to grayscale (cv2.cvtColor). It is then thresholded using cv2.threshold at a value of 80 to create a binary image.
- **Contour Filtering:** Contours are identified using cv2.findContours. The script iterates through the contours, keeping only those with an area ≥ 10 or a bounding box dimension ≥ 6.
- **Masking:** A mask is created by drawing these filtered contours and applying dilation (cv2.dilate) using a 2×2 kernel. The original image pixels are retained only where the mask > 0.
- **Final Denoising:** The isolated subject is smoothed using cv2.fastNlMeansDenoisingColored(clean, None, 3, 3, 7, 21).

VI. RESULTS AND OBSERVATION

The final algorithms successfully generated clean, enhanced images.

- **Scenery Output:** The noisy.jpg (65 KB) was successfully transformed into enhanced_landscape.jpg (59 KB). The bilateral filtering effectively preserved the edges while the gamma correction boosted the contrast of the shadowed regions.
- **Iron-Man Output:** The iron_man_noisy.jpg (175 KB) was successfully transformed into iron_man_clean.jpg (108 KB). By using contour area and bounding box parameters, the subject was successfully separated from the background noise.

A drawback of the contour-masking approach used for the Iron-Man image is its strict dependency on hardcoded threshold values (80 for binary, 10 for area, 6 for bounding box), which may not generalize well to images with different lighting or scaling.

VII. FUTURE WORK

Future work involves addressing the limitations of hardcoded thresholds. Adaptive thresholding could be implemented to make the Iron-Man contour extraction robust against varying lighting conditions. Additionally, testing the pipeline's execution time on actual drone hardware (like a Raspberry Pi or Jetson Nano) is necessary to ensure these algorithms can run at the required frame rates for real-time video feeds.

CONCLUSION

The problem involved restoring two heavily degraded images to recover usable visual data. By implementing a combination of bilateral filtering, Non-Local Means denoising, contour extraction, and gamma correction, the noise was successfully eliminated while preserving vital edges and colors. This pipeline provides a net output of clean, highly legible images that are extremely useful for ARK, as they serve as a necessary preprocessing step to ensure high accuracy in real-time object tracking and navigation systems.